AI Assisted Coding

Assignment 12.5

Name: R.Bharadwaj

Hall ticket no: 2303A51610

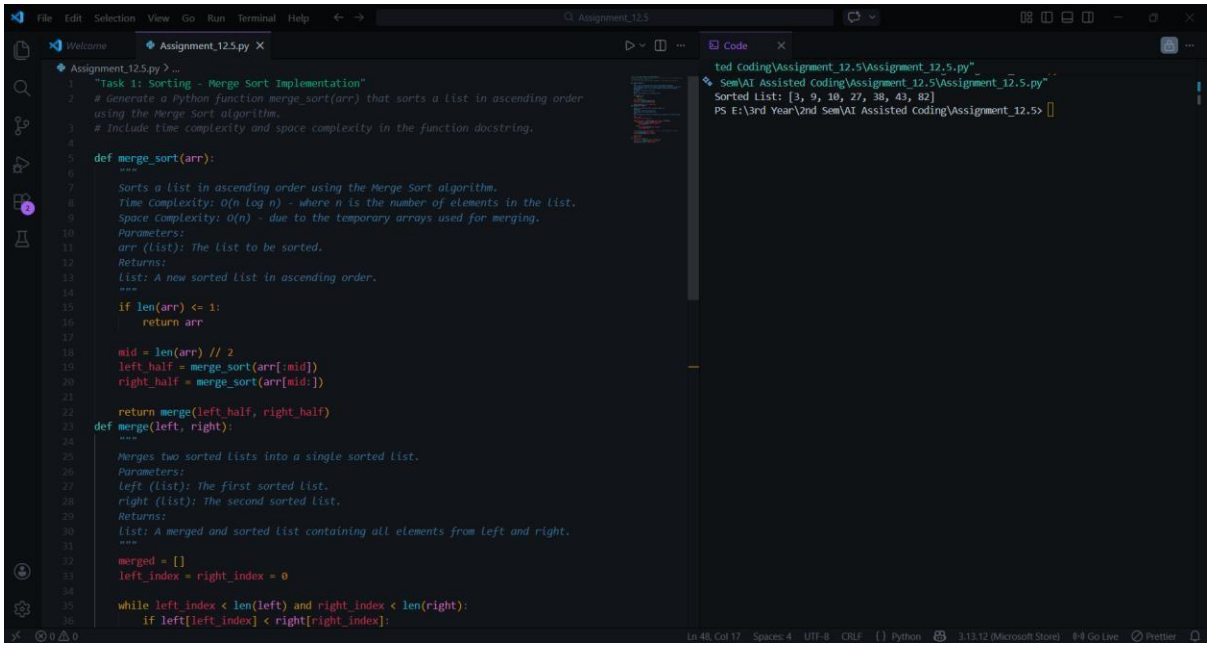
Batch no: 19

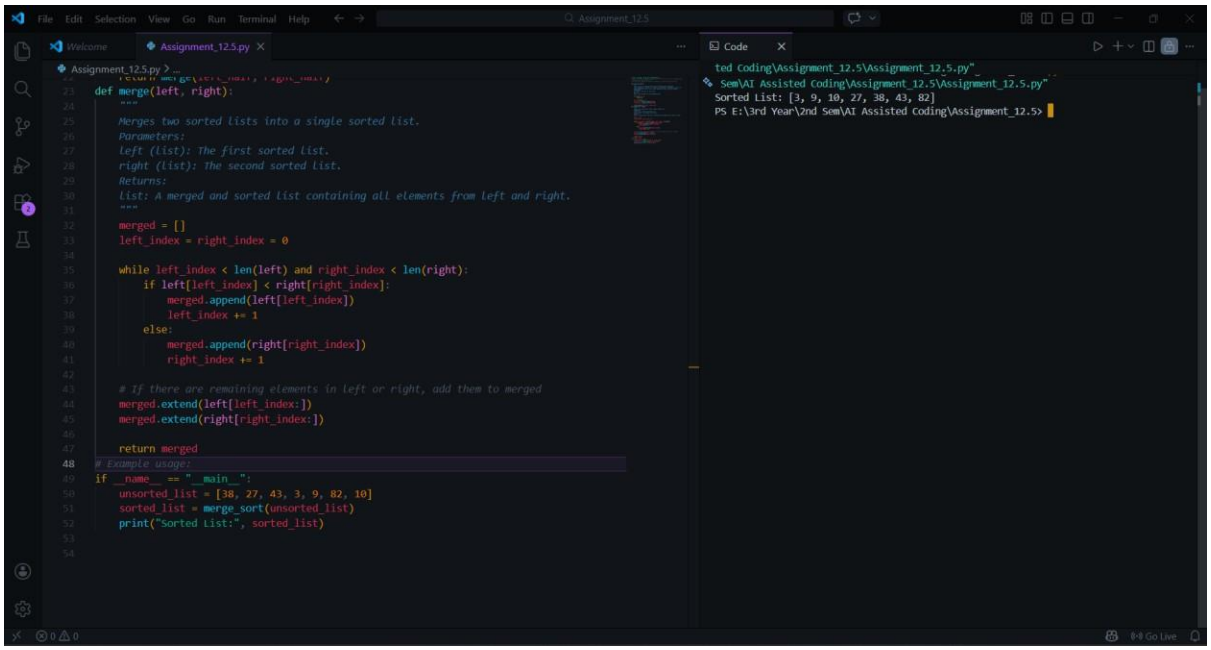
Task 1: Sorting – Merge Sort Implementation

Prompt:

Generate a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm.
Include time complexity and space complexity in the function docstring.

Code & Output:





Explanation:

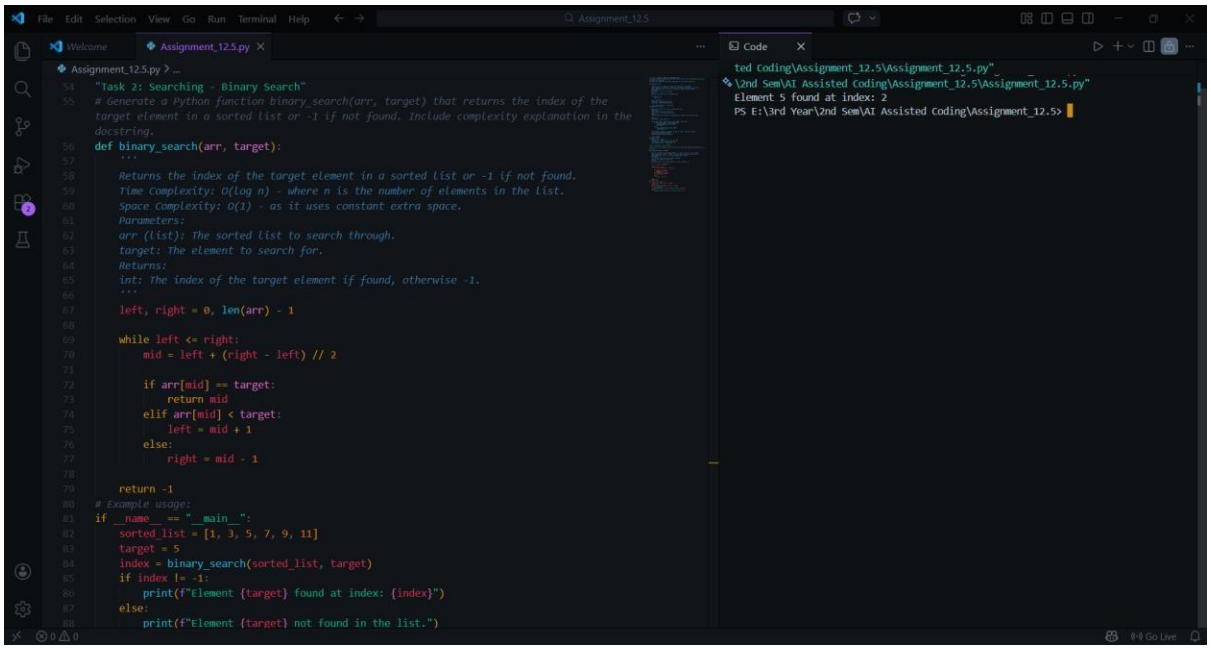
Merge Sort divides the list into smaller halves recursively and then merges them in sorted order. It guarantees O(n log n) time complexity in all cases, making it efficient for large datasets. The algorithm requires additional memory for merging, resulting in O(n) space complexity. AI assistance helped generate structured recursive logic with proper documentation.

Task 2: Searching – Binary Search

Prompt:

Generate a Python function binary_search(arr, target) that returns the index of the target element in a sorted list or -1 if not found. Include complexity explanation in the docstring.

Code & Output:



Explanation:

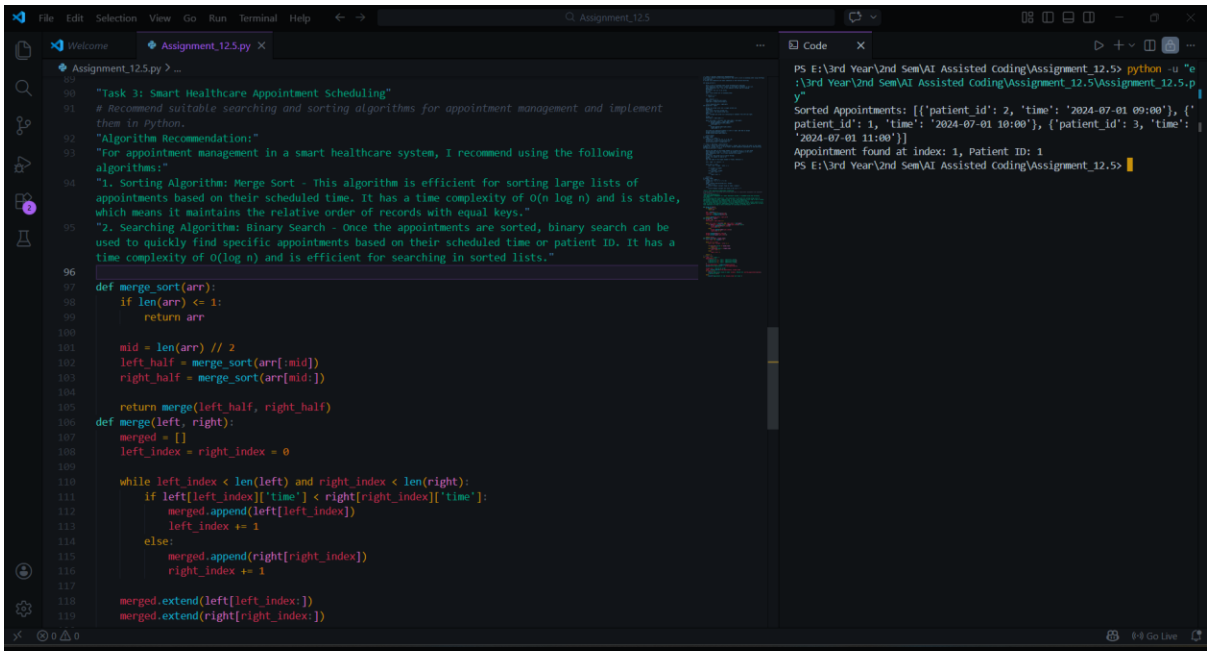
Binary Search works by repeatedly dividing the sorted list into halves. It significantly reduces the search space, resulting in logarithmic time complexity. It is efficient compared to linear search for large sorted datasets. AI assistance helped structure the iterative logic and complexity documentation clearly.

Task 3: Smart Healthcare Appointment Scheduling

Prompt:

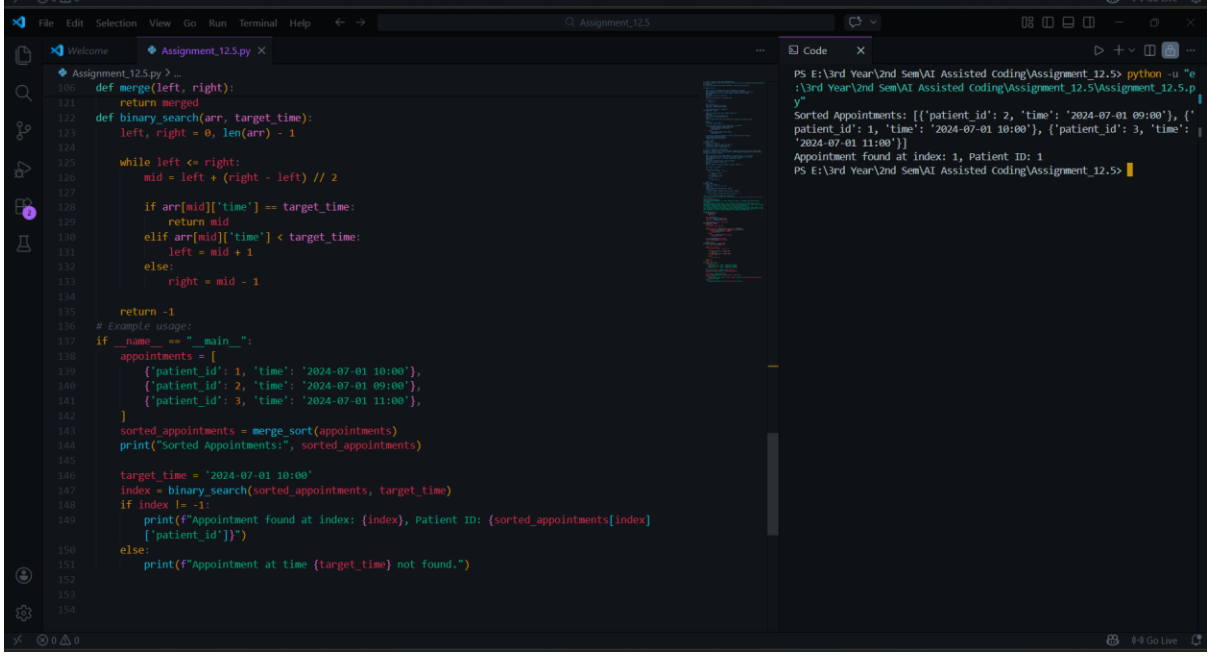
Recommend suitable searching and sorting algorithms for appointment management and implement them in Python.

Code & Output:



```python
"Task 3: Smart Healthcare Appointment Scheduling"
# Recommend suitable searching and sorting algorithms for appointment management and implement them in Python.
"Algorithm Recommendation:"
"For appointment management in a smart healthcare system, I recommend using the following algorithms:"
"1. Sorting Algorithm: Merge Sort - This algorithm is efficient for sorting large lists of appointments based on their scheduled time. It has a time complexity of O(n log n) and is stable, which means it maintains the relative order of records with equal keys."
"2. Searching Algorithm: Binary Search - Once the appointments are sorted, binary search can be used to quickly find specific appointments based on their scheduled time or patient ID. It has a time complexity of O(log n) and is efficient for searching in sorted lists."

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index]['time'] < right[right_index]['time']:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e
:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.p
y"
Sorted Appointments: [{'patient_id': 2, 'time': '2024-07-01 09:00'}, {'
patient_id': 1, 'time': '2024-07-01 10:00'}, {'patient_id': 3, 'time':
'2024-07-01 11:00'}]
Appointment found at index: 1, Patient ID: 1
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```



```python
def merge(left, right):
    return merged
def binary_search(arr, target_time):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid]['time'] == target_time:
            return mid
        elif arr[mid]['time'] < target_time:
            left = mid + 1
        else:
            right = mid - 1

    return -1
# Example usage:
if __name__ == "__main__":
    appointments = [
        {'patient_id': 1, 'time': '2024-07-01 10:00'},
        {'patient_id': 2, 'time': '2024-07-01 09:00'},
        {'patient_id': 3, 'time': '2024-07-01 11:00'},
    ]
    sorted_appointments = merge_sort(appointments)
    print("Sorted Appointments:", sorted_appointments)

    target_time = '2024-07-01 10:00'
    index = binary_search(sorted_appointments, target_time)
    if index != -1:
        print(f"Appointment found at index: {index}, Patient ID: {sorted_appointments[index]
        ['patient_id']}")
    else:
        print(f"Appointment at time {target_time} not found.")
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e
:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.p
y"
Sorted Appointments: [{'patient_id': 2, 'time': '2024-07-01 09:00'}, {'
patient_id': 1, 'time': '2024-07-01 10:00'}, {'patient_id': 3, 'time':
'2024-07-01 11:00'}]
Appointment found at index: 1, Patient ID: 1
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Explanation:

Binary Search is efficient for searching appointment IDs when records are sorted. Merge Sort ensures stable and efficient sorting by time or fee. These algorithms provide optimal performance for medium to large datasets. AI-assisted reasoning supported algorithm selection based on efficiency requirements.

Task 4: Railway Ticket Reservation System

Prompt:

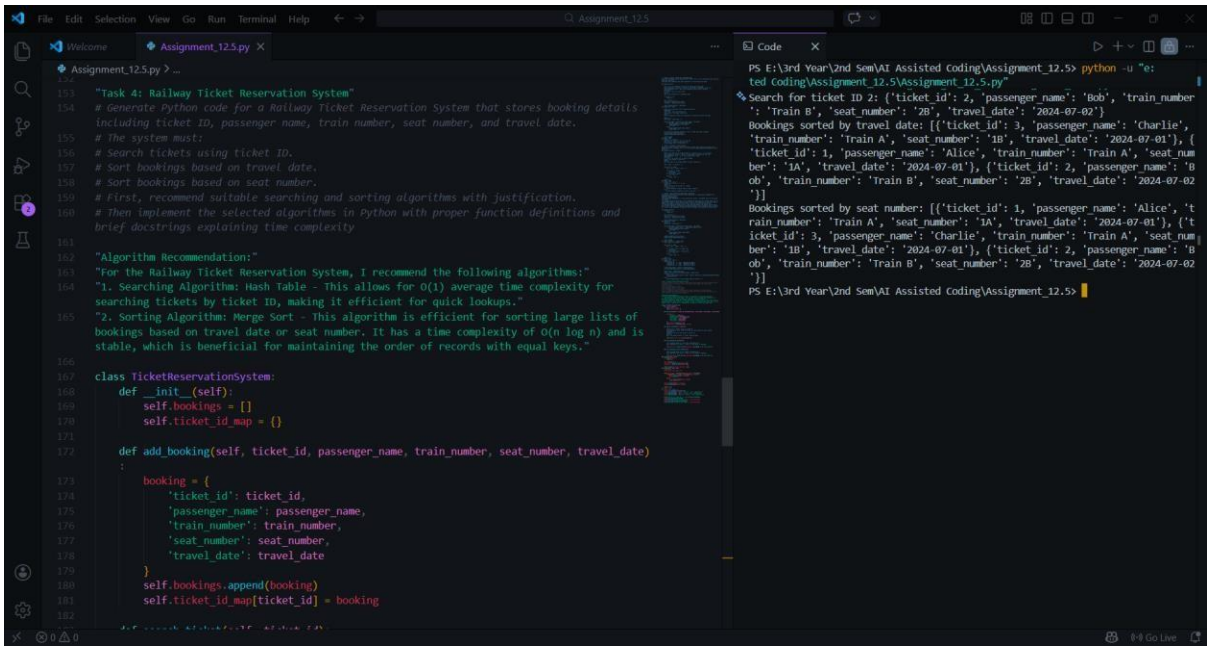
Generate Python code for a Railway Ticket Reservation System that stores booking details including ticket ID, passenger name, train number, seat number, and travel date.

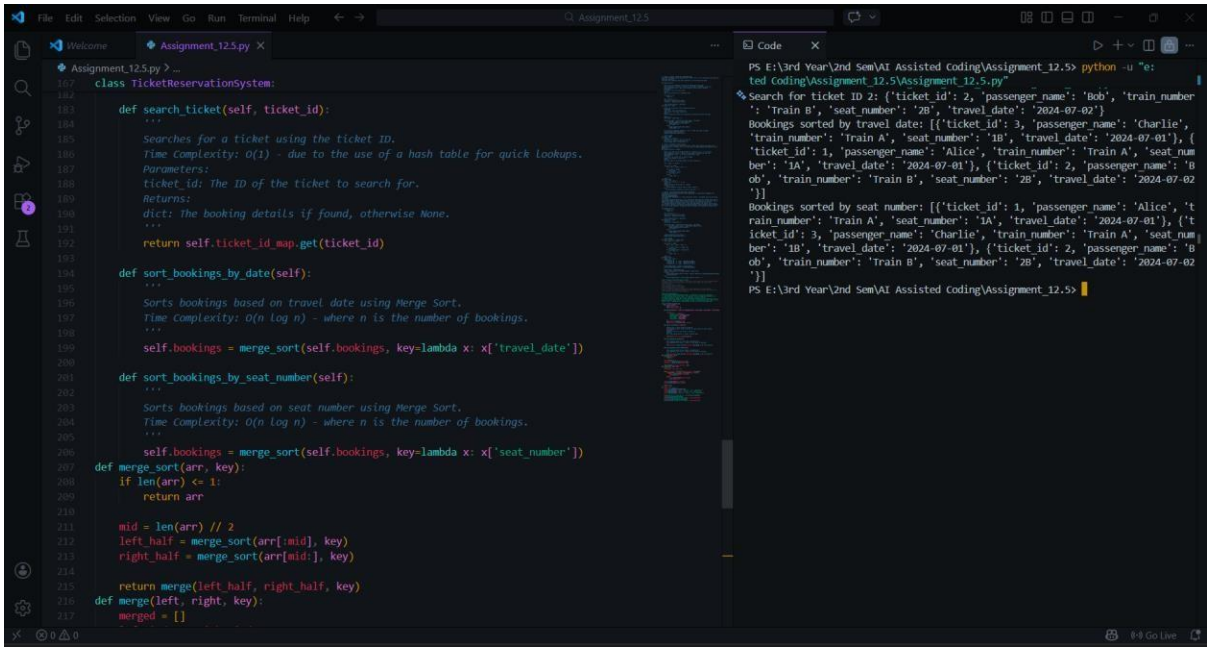
The system must:

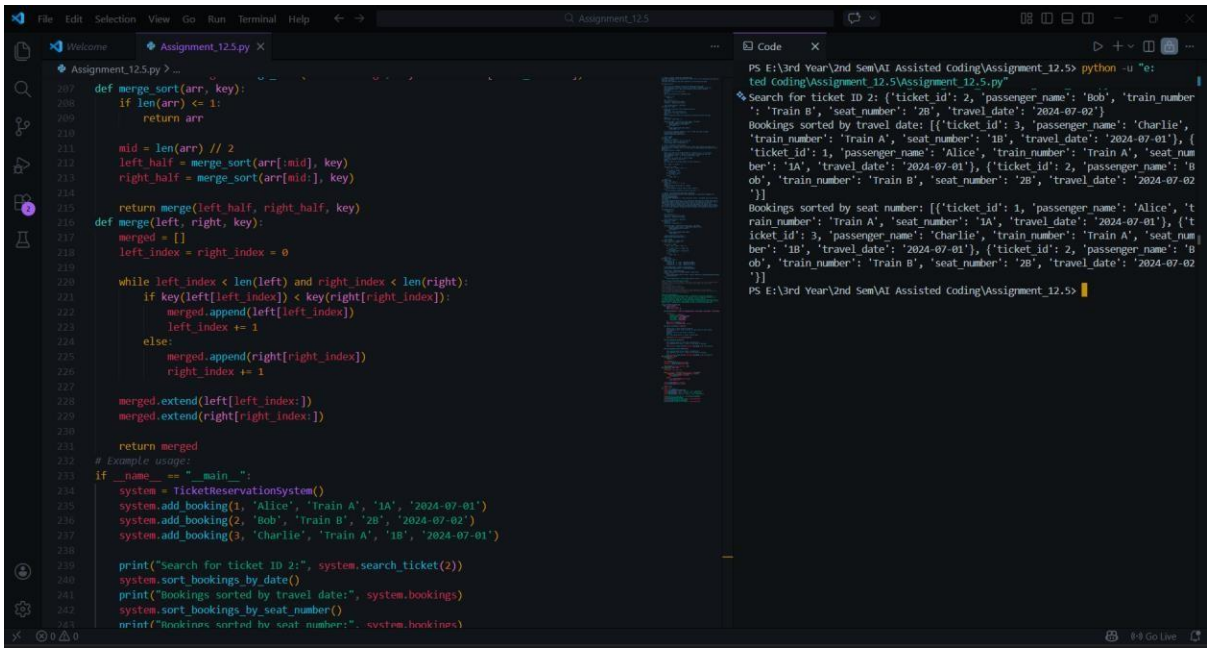
1. Search tickets using ticket ID.
2. Sort bookings based on travel date.
3. Sort bookings based on seat number.

First, recommend suitable searching and sorting algorithms with justification.
Then implement the selected algorithms in Python with proper function definitions and brief docstrings explaining time complexity.

Code & Output:







```python
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example usage:
if __name__ == "__main__":
    system = TicketReservationSystem()
    system.add_booking(1, 'Alice', 'Train A', '1A', '2024-07-01')
    system.add_booking(2, 'Bob', 'Train B', '2B', '2024-07-02')
    system.add_booking(3, 'Charlie', 'Train A', '1B', '2024-07-01')

    print("Search for ticket ID 2:", system.search_ticket(2))
    system.sort_bookings_by_date()
    print("Bookings sorted by travel date:", system.bookings)
    system.sort_bookings_by_seat_number()
    print("Bookings sorted by seat number:", system.bookings)
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:
ted Coding\Assignment_12.5\Assignment_12.5.py"
Search for ticket ID 2: {'ticket_id': 2, 'passenger_name': 'Bob', 'train_number
': 'Train B', 'seat_number': '2B', 'travel_date': '2024-07-02'}
Bookings sorted by travel date: [{'ticket_id': 3, 'passenger_name': 'Charlie',
'train_number': 'Train A', 'seat_number': '1B', 'travel_date': '2024-07-01'}, {
'ticket_id': 1, 'passenger_name': 'Alice', 'train_number': 'Train A', 'seat_num
ber': '1A', 'travel_date': '2024-07-01'}, {'ticket_id': 2, 'passenger_name': 'B
ob', 'train_number': 'Train B', 'seat_number': '2B', 'travel_date': '2024-07-02
'}]
Bookings sorted by seat number: [{'ticket_id': 1, 'passenger_name': 'Alice', 't
rain_number': 'Train A', 'seat_number': '1A', 'travel_date': '2024-07-01'}, {'t
icket_id': 3, 'passenger_name': 'Charlie', 'train_number': 'Train A', 'seat_num
ber': '1B', 'travel_date': '2024-07-01'}, {'ticket_id': 2, 'passenger_name': 'B
ob', 'train_number': 'Train B', 'seat_number': '2B', 'travel_date': '2024-07-02
'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Explanation:

Binary Search is chosen for efficient ticket lookup when records are sorted by ticket ID, reducing search time to O(log n). Sorting is done using a stable sorting method (similar to Merge Sort Behaviour) to organize bookings by date or seat number. This ensures structured record management. AI assistance helped select efficient algorithms and implement clean functions.

Task 5: Smart Hostel Room Allocation System

Prompt:

Generate Python code for a Smart Hostel Room Allocation System storing student_id, room_number, floor, and allocation_date.
The system must:

- Search allocation details using student_id
- Sort records based on room_number
- Sort records based on allocation_date

Recommend suitable algorithms and implement them with brief complexity documentation.

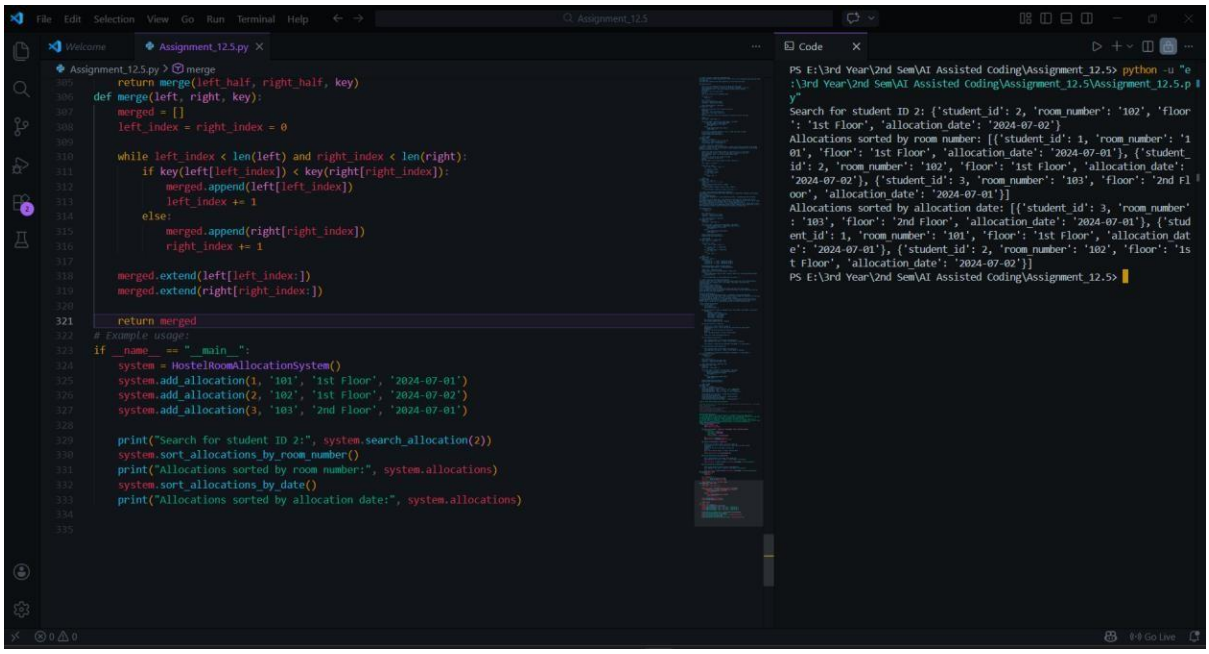
Code & Output:

```python
"Task 5: Smart Hostel Room Allocation System"

# Generate Python code for a Smart Hostel Room Allocation System storing student_id, room_number,
floor, and allocation_date.
# The system must:
# Search allocation details using student_id
# Sort records based on room_number
# Sort records based on allocation_date
# Recommend suitable algorithms and implement them with brief complexity documentation.

"Algorithm Recommendation:"
"For the Smart Hostel Room Allocation System, I recommend the following algorithms:"
"1. Searching Algorithm: Hash Table - This allows for O(1) average time complexity for searching
allocation details by student_id, making it efficient for quick lookups."
"2. Sorting Algorithm: Merge Sort - This algorithm is efficient for sorting large lists of
allocations based on room_number or allocation_date. It has a time complexity of O(n log n) and is
stable, which is beneficial for maintaining the order of records with equal keys."
class HostelRoomAllocationSystem:
    def __init__(self):
        self.allocations = []
        self.student_id_map = {}

    def add_allocation(self, student_id, room_number, floor, allocation_date):
        allocation = {
            'student_id': student_id,
            'room_number': room_number,
            'floor': floor,
            'allocation_date': allocation_date
        }
        self.allocations.append(allocation)
        self.student_id_map[student_id] = allocation

    def search_allocation(self, student_id):
        '''
        Searches for allocation details using the student ID.
        Time Complexity: O(1) - due to the use of a hash table for quick lookups.
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e
:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.p
y"
Search for student ID 2: {'student_id': 2, 'room_number': '102', 'floor
': '1st Floor', 'allocation_date': '2024-07-02'}
Allocations sorted by room number: [{'student_id': 1, 'room_number': '1
01', 'floor': '1st Floor', 'allocation_date': '2024-07-01'}, {'student_
id': 2, 'room_number': '102', 'floor': '1st Floor', 'allocation_date':
'2024-07-02'}, {'student_id': 3, 'room_number': '103', 'floor': '2nd Fl
oor', 'allocation_date': '2024-07-01'}]
Allocations sorted by allocation date: [{'student_id': 3, 'room_number'
: '103', 'floor': '2nd Floor', 'allocation_date': '2024-07-01'}, {'stud
ent_id': 1, 'room_number': '101', 'floor': '1st Floor', 'allocation_dat
e': '2024-07-01'}, {'student_id': 2, 'room_number': '102', 'floor': '1s
t Floor', 'allocation_date': '2024-07-02'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

```python
class HostelRoomAllocationSystem:
    def search_allocation(self, student_id):
        Parameters:
        student_id: The ID of the student to search for.
        Returns:
        dict: The allocation details if found, otherwise None.
        '''
        return self.student_id_map.get(student_id)

    def sort_allocations_by_room_number(self):
        '''
        Sorts records based on room number using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of allocations.
        '''
        self.allocations = merge_sort(self.allocations, key=lambda x: x['room_number'])

    def sort_allocations_by_date(self):
        '''
        Sorts records based on allocation date using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of allocations.
        '''
        self.allocations = merge_sort(self.allocations, key=lambda x: x['allocation_date'])
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e
:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.p
y"
Search for student ID 2: {'student_id': 2, 'room_number': '102', 'floor
': '1st Floor', 'allocation_date': '2024-07-02'}
Allocations sorted by room number: [{'student_id': 1, 'room_number': '1
01', 'floor': '1st Floor', 'allocation_date': '2024-07-01'}, {'student_
id': 2, 'room_number': '102', 'floor': '1st Floor', 'allocation_date':
'2024-07-02'}, {'student_id': 3, 'room_number': '103', 'floor': '2nd Fl
oor', 'allocation_date': '2024-07-01'}]
Allocations sorted by allocation date: [{'student_id': 3, 'room_number'
: '103', 'floor': '2nd Floor', 'allocation_date': '2024-07-01'}, {'stud
ent_id': 1, 'room_number': '101', 'floor': '1st Floor', 'allocation_dat
e': '2024-07-01'}, {'student_id': 2, 'room_number': '102', 'floor': '1s
t Floor', 'allocation_date': '2024-07-02'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```



```python
    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example usage:
if __name__ == "__main__":
    system = HostelRoomAllocationSystem()
    system.add_allocation(1, '101', '1st Floor', '2024-07-01')
    system.add_allocation(2, '102', '1st Floor', '2024-07-02')
    system.add_allocation(3, '103', '2nd Floor', '2024-07-01')

    print("Search for student ID 2:", system.search_allocation(2))
    system.sort_allocations_by_room_number()
    print("Allocations sorted by room number:", system.allocations)
    system.sort_allocations_by_date()
    print("Allocations sorted by allocation date:", system.allocations)
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e
:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.p
y"
Search for student ID 2: {'student_id': 2, 'room_number': '102', 'floor
': '1st Floor', 'allocation_date': '2024-07-02'}
Allocations sorted by room number: [{'student_id': 1, 'room_number': '1
01', 'floor': '1st Floor', 'allocation_date': '2024-07-01'}, {'student_
id': 2, 'room_number': '102', 'floor': '1st Floor', 'allocation_date':
'2024-07-02'}, {'student_id': 3, 'room_number': '103', 'floor': '2nd Fl
oor', 'allocation_date': '2024-07-01'}]
Allocations sorted by allocation date: [{'student_id': 3, 'room_number'
: '103', 'floor': '2nd Floor', 'allocation_date': '2024-07-01'}, {'stud
ent_id': 1, 'room_number': '101', 'floor': '1st Floor', 'allocation_dat
e': '2024-07-01'}, {'student_id': 2, 'room_number': '102', 'floor': '1s
t Floor', 'allocation_date': '2024-07-02'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Explanation:

Binary Search allows fast lookup of student allocation records when sorted by student ID. Sorting by room number or allocation date organizes hostel data efficiently. These algorithms ensure scalability for large datasets. AI guidance supported correct structure and optimized implementation.

Task 6: Online Movie Streaming Platform

Prompt:

Generate Python code for a Movie Streaming Platform storing movie_id, title, genre, rating, and release_year.
The system must:

- Search movies by movie_id
- Sort movies based on rating
- Sort movies based on release_year

Recommend suitable algorithms and implement them.

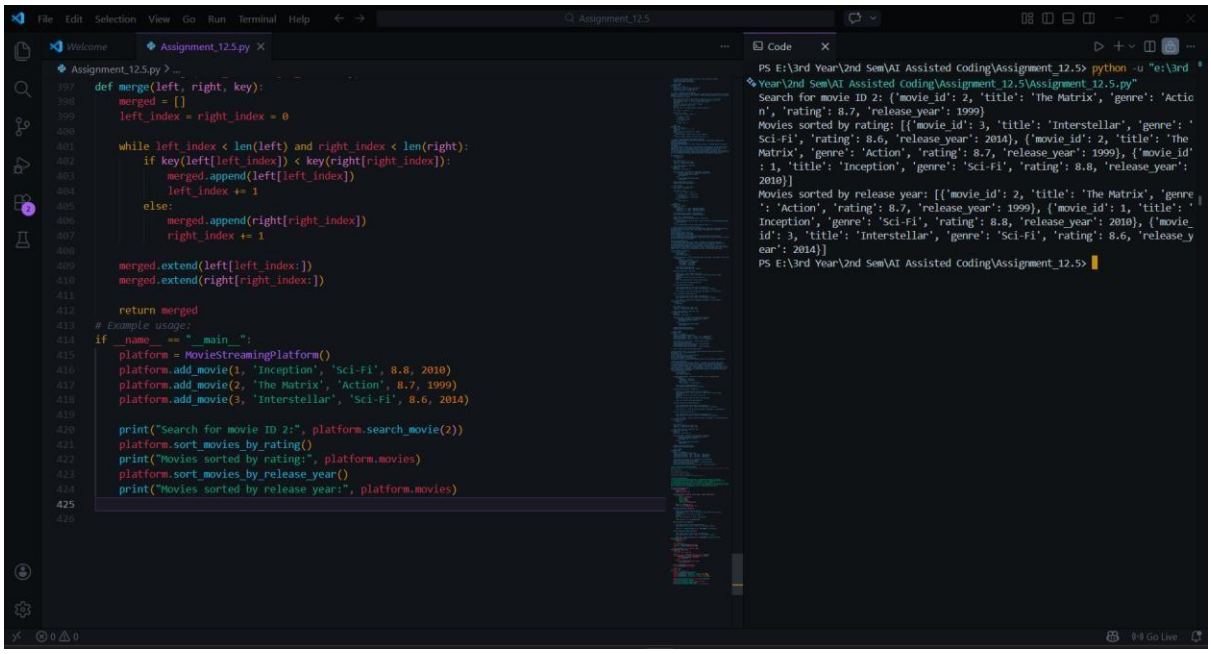
Code & Output:

```python
"Task 6: Online Movie Streaming Platform"
# Generate Python code for a Movie Streaming Platform storing movie_id, title, genre, rating,
and release_year.
# The system must:
# Search movies by movie_id
# Sort movies based on rating
# Sort movies based on release_year
# Recommend suitable algorithms and implement them.

"Algorithm Recommendation:"
"For the Online Movie Streaming Platform, I recommend the following algorithms:"
"1. Searching Algorithm: Hash Table - This allows for O(1) average time complexity for
searching movies by movie_id, making it efficient for quick lookups."
"2. Sorting Algorithm: Merge Sort - This algorithm is efficient for sorting large lists of
movies based on rating or release_year. It has a time complexity of O(n log n) and is stable,
which is beneficial for maintaining the order of records with equal keys."

class MovieStreamingPlatform:
    def __init__(self):
        self.movies = []
        self.movie_id_map = {}

    def add_movie(self, movie_id, title, genre, rating, release_year):
        movie = {
            'movie_id': movie_id,
            'title': title,
            'genre': genre,
            'rating': rating,
            'release_year': release_year
        }
        self.movies.append(movie)
        self.movie_id_map[movie_id] = movie

    def search_movie(self, movie_id):
        '''
        Searches for a movie using the movie ID.
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd
Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for movie ID 2: {'movie_id': 2, 'title': 'The Matrix', 'genre': 'Actio
n', 'rating': 8.7, 'release_year': 1999}
Movies sorted by rating: [{'movie_id': 3, 'title': 'Interstellar', 'genre': '
Sci-Fi', 'rating': 8.6, 'release_year': 2014}, {'movie_id': 2, 'title': 'The
Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id'
: 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year':
2010}]
Movies sorted by release year: [{'movie_id': 2, 'title': 'The Matrix', 'genre
': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id': 1, 'title': '
Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}, {'movie_
id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_y
ear': 2014}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

```python
class MovieStreamingPlatform:
    def search_movie(self, movie_id):
        '''
        Searches for a movie using the movie ID.
        Time Complexity: O(1) - due to the use of a hash table for quick lookups.
        Parameters:
        movie_id: The ID of the movie to search for.
        Returns:
        dict: The movie details if found, otherwise None.
        '''
        return self.movie_id_map.get(movie_id)

    def sort_movies_by_rating(self):
        '''
        Sorts movies based on rating using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of movies.
        '''
        self.movies = merge_sort(self.movies, key=lambda x: x['rating'])

    def sort_movies_by_release_year(self):
        '''
        Sorts movies based on release year using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of movies.
        '''
        self.movies = merge_sort(self.movies, key=lambda x: x['release_year'])
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd
Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for movie ID 2: {'movie_id': 2, 'title': 'The Matrix', 'genre': 'Actio
n', 'rating': 8.7, 'release_year': 1999}
Movies sorted by rating: [{'movie_id': 3, 'title': 'Interstellar', 'genre': '
Sci-Fi', 'rating': 8.6, 'release_year': 2014}, {'movie_id': 2, 'title': 'The
Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id'
: 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year':
2010}]
Movies sorted by release year: [{'movie_id': 2, 'title': 'The Matrix', 'genre
': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id': 1, 'title': '
Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}, {'movie_
id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_y
ear': 2014}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```



```
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example usage:
if __namo__ == "__main__":
    platform = MovieStreamingPlatform()
    platform.add_movie(1, 'Inception', 'Sci-Fi', 8.8, 2010)
    platform.add_movie(2, 'The Matrix', 'Action', 8.7, 1999)
    platform.add_movie(3, 'Interstellar', 'Sci-Fi', 8.6, 2014)

    print("Search for movie ID 2:", platform.search_movie(2))
    platform.sort_movies_by_rating()
    print("Movies sorted by rating:", platform.movies)
    platform.sort_movies_by_release_year()
    print("Movies sorted by release year:", platform.movies)
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd
Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for movie ID 2: {'movie_id': 2, 'title': 'The Matrix', 'genre': 'Actic
n', 'rating': 8.7, 'release_year': 1999}
Movies sorted by rating: [{'movie_id': 3, 'title': 'Interstellar', 'genre': '
Sci-Fi', 'rating': 8.6, 'release_year': 2014}, {'movie_id': 2, 'title': 'The
Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id'
: 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year':
2010}]
Movies sorted by release year: [{'movie_id': 2, 'title': 'The Matrix', 'genre
': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id': 1, 'title': '
Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}, {'movie_
id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_y
ear': 2014}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Explanation:

Binary Search is suitable for quick movie lookup when sorted by movie ID. Sorting by rating helps rank movies, while sorting by release year organizes content chronologically. These algorithms ensure efficient data management for streaming platforms. AI assistance improved modularity and clarity.

Task 7: Smart Agriculture Crop Monitoring System

Prompt:

Generate Python code for a Crop Monitoring System storing crop_id, crop_name, soil_moisture_level, temperature, and yield_estimate.
The system must:

- Search crop details using crop_id
- Sort crops based on soil_moisture_level
- Sort crops based on yield_estimate

Recommend suitable algorithms and implement them.

Code & Output:

Assignment_12.5.py > ...

```python
"Task 7: Smart Agriculture Crop Monitoring System"
# Generate Python code for a Crop Monitoring System storing crop_id, crop_name, soil_moisture_level, temperature, and yield_estimate.
# The system must:
# Search crop details using crop_id
# Sort crops based on soil_moisture_level
# Sort crops based on yield_estimate
# Recommend suitable algorithms and implement them.
"Algorithm Recommendation:"
"For the Smart Agriculture Crop Monitoring System, I recommend the following algorithms:"
"1. Searching Algorithm: Hash Table - This allows for O(1) average time complexity for searching crop details by crop_id, making it efficient for quick lookups."
"2. Sorting Algorithm: Merge Sort - This algorithm is efficient for sorting large lists of crops based on soil_moisture_level or yield_estimate. It has a time complexity of O(n log n) and is stable, which is beneficial for maintaining the order of records with equal keys."
class CropMonitoringSystem:
    def __init__(self):
        self.crops = []
        self.crop_id_map = {}

    def add_crop(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
        crop = {
            'crop_id': crop_id,
            'crop_name': crop_name,
            'soil_moisture_level': soil_moisture_level,
            'temperature': temperature,
            'yield_estimate': yield_estimate
        }
        self.crops.append(crop)
        self.crop_id_map[crop_id] = crop

    def search_crop(self, crop_id):
        '''
        Searches for crop details using the crop ID.
        Time Complexity: O(1) - due to the use of a hash table for quick lookups.
        Parameters:
        crop_id: The ID of the crop to search for.
```

Code

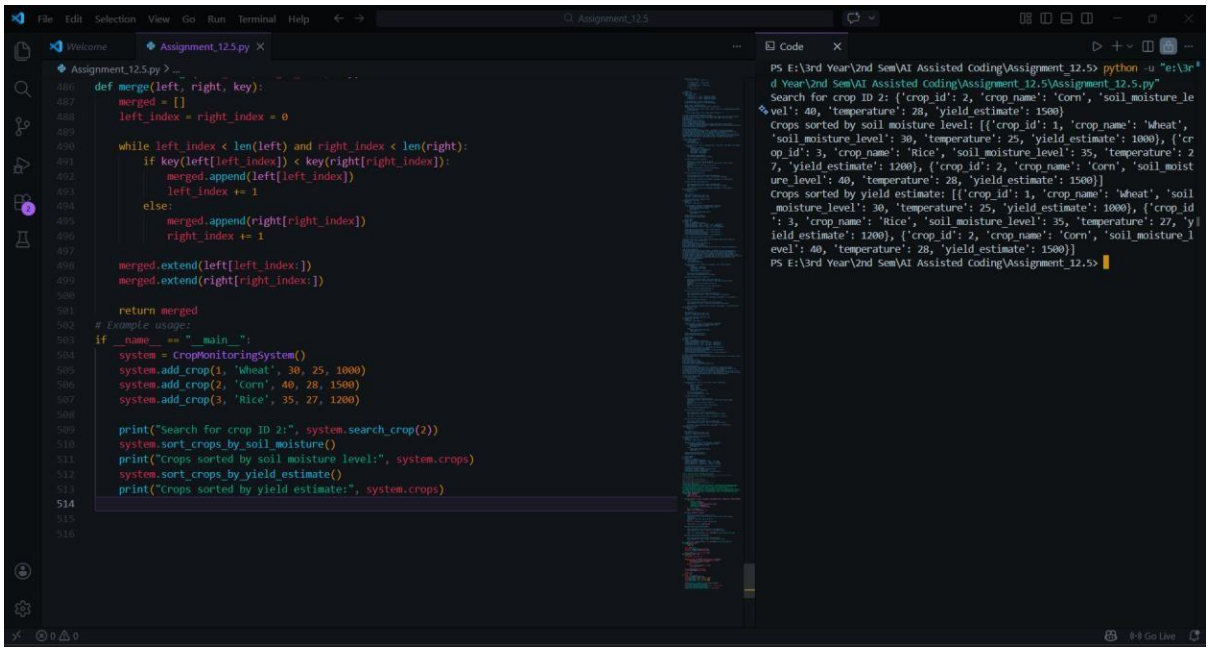
```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for crop ID 2: {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}
Crops sorted by soil moisture level: [{'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}, {'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 35, 'temperature': 27, 'yield_estimate': 1200}, {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}]
Crops sorted by yield estimate: [{'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}, {'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 35, 'temperature': 27, 'yield_estimate': 1200}, {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Assignment_12.5.py > ...

```python
class CropMonitoringSystem:
    def search_crop(self, crop_id):
        '''
        Searches for crop details using the crop ID.
        Time Complexity: O(1) - due to the use of a hash table for quick lookups.
        Parameters:
        crop_id: The ID of the crop to search for.
        Returns:
        dict: The crop details if found, otherwise None.
        '''
        return self.crop_id_map.get(crop_id)

    def sort_crops_by_soil_moisture(self):
        '''
        Sorts crops based on soil moisture level using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of crops.
        '''
        self.crops = merge_sort(self.crops, key=lambda x: x['soil_moisture_level'])

    def sort_crops_by_yield_estimate(self):
        '''
        Sorts crops based on yield estimate using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of crops.
        '''
        self.crops = merge_sort(self.crops, key=lambda x: x['yield_estimate'])
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0
```

Code

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for crop ID 2: {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}
Crops sorted by soil moisture level: [{'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}, {'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 35, 'temperature': 27, 'yield_estimate': 1200}, {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}]
Crops sorted by yield estimate: [{'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}, {'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 35, 'temperature': 27, 'yield_estimate': 1200}, {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```



Explanation:

Binary Search enables efficient crop lookup after sorting by crop ID. Sorting by moisture or yield estimate helps farmers analyze data effectively. These algorithms ensure optimized performance for agricultural monitoring systems. AI-supported reasoning guided appropriate algorithm selection.

Task 8: Airport Flight Management System

Prompt:

Generate Python code for an Airport Flight Management System storing flight_id, airline_name, departure_time, arrival_time, and status.
The system must:

- Search flight details using flight_id
- Sort flights based on departure_time
- Sort flights based on arrival_time

Recommend suitable algorithms and implement them.

Code & Output:

Assignment_12.5.py

```python
"Task 8: Airport Flight Management System"
# Generate Python code for an Airport Flight Management System storing flight_id, airline_name, departure_time, arrival_time, and status.
# The system must:
# Search flight details using flight_id
# Sort flights based on departure_time
# Sort flights based on arrival_time
# Recommend suitable algorithms and implement them.
"Algorithm Recommendation:"
"For the Airport Flight Management System, I recommend the following algorithms:"
"1. Searching Algorithm: Hash Table - This allows for O(1) average time complexity for searching flight details by flight_id, making it efficient for quick lookups."
"2. Sorting Algorithm: Merge Sort - This algorithm is efficient for sorting large lists of flights based on departure_time or arrival_time. It has a time complexity of O(n log n) and is stable, which is beneficial for maintaining the order of records with equal keys."
class FlightManagementSystem:
    def __init__(self):
        self.flights = []
        self.flight_id_map = {}

    def add_flight(self, flight_id, airline_name, departure_time, arrival_time, status):
        flight = {
            'flight_id': flight_id,
            'airline_name': airline_name,
            'departure_time': departure_time,
            'arrival_time': arrival_time,
            'status': status
        }
        self.flights.append(flight)
        self.flight_id_map[flight_id] = flight

    def search_flight(self, flight_id):
        '''
```

Code

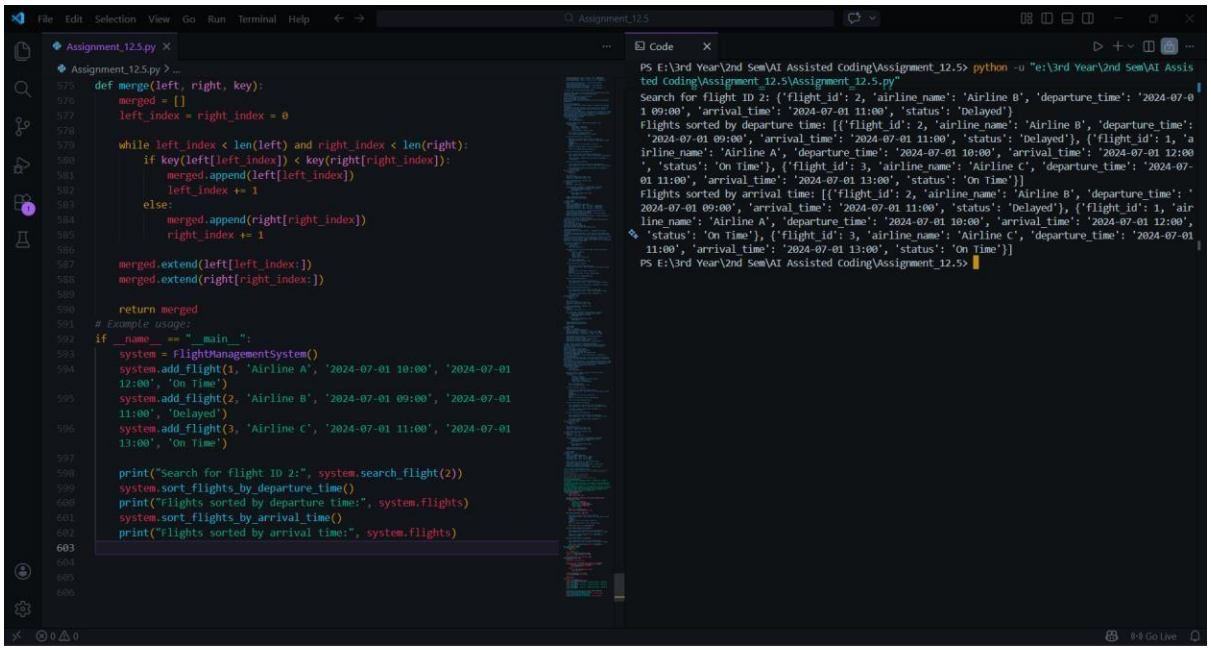
```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for flight ID 2: {'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}
Flights sorted by departure time: [{'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}, {'flight_id': 1, 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 3, 'airline_name': 'Airline C', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'On Time'}]
Flights sorted by arrival time: [{'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}, {'flight_id': 1, 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 3, 'airline_name': 'Airline C', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'On Time'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Assignment_12.5.py

```python
class FlightManagementSystem:
    def search_flight(self, flight_id):
        '''
        Searches for flight details using the flight ID.
        Time Complexity: O(1) - due to the use of a hash table for quick lookups.
        Parameters:
        flight_id: The ID of the flight to search for.
        Returns:
        dict: The flight details if found, otherwise None.
        '''
        return self.flight_id_map.get(flight_id)

    def sort_flights_by_departure_time(self):
        '''
        Sorts flights based on departure time using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of flights.
        '''
        self.flights = merge_sort(self.flights, key=lambda x: x['departure_time'])

    def sort_flights_by_arrival_time(self):
        '''
        Sorts flights based on arrival time using Merge Sort.
        Time Complexity: O(n log n) - where n is the number of flights.
        '''
        self.flights = merge_sort(self.flights, key=lambda x: x['arrival_time'])
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
```

Code

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for flight ID 2: {'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}
Flights sorted by departure time: [{'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}, {'flight_id': 1, 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 3, 'airline_name': 'Airline C', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'On Time'}]
Flights sorted by arrival time: [{'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}, {'flight_id': 1, 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 3, 'airline_name': 'Airline C', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'On Time'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```



```python
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example usage:
if __name__ == "__main__":
    system = FlightManagementSystem()
    system.add_flight(1, 'Airline A', '2024-07-01 10:00', '2024-07-01 12:00', 'On Time')
    system.add_flight(2, 'Airline B', '2024-07-01 09:00', '2024-07-01 11:00', 'Delayed')
    system.add_flight(3, 'Airline C', '2024-07-01 11:00', '2024-07-01 13:00', 'On Time')

    print("Search for flight ID 2:", system.search_flight(2))
    system.sort_flights_by_departure_time()
    print("Flights sorted by departure time:", system.flights)
    system.sort_flights_by_arrival_time()
    print("Flights sorted by arrival time:", system.flights)
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5\Assignment_12.5.py"
Search for flight ID 2: {'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}
Flights sorted by departure time: [{'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}, {'flight_id': 1, 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 3, 'airline_name': 'Airline C', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'On Time'}]
Flights sorted by arrival time: [{'flight_id': 2, 'airline_name': 'Airline B', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'Delayed'}, {'flight_id': 1, 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 3, 'airline_name': 'Airline C', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'On Time'}]
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_12.5>
```

Explanation:

Binary Search ensures fast flight lookup when sorted by flight ID. Sorting by departure or arrival time organizes airport schedules effectively. These algorithms provide efficient record management. AI assistance supported correct algorithm implementation and modular function design.

Final Conclusion:

This lab demonstrated the implementation of sorting and searching algorithms using AI assistance across multiple real-world scenarios. Binary Search was selected for efficient searching due to its logarithmic time complexity, while Merge Sort was chosen for stable and reliable sorting performance. AI tools supported algorithm recommendation, code generation, and optimization. However, understanding algorithm efficiency and selecting appropriate techniques remain the developer’s responsibility to ensure correct and scalable solutions.